

First Look at Django Forms (Chai aur Code)

Complete Quick Revision Notes • Forms Wiring, Validation & Template Rendering in Hinglish

Django Tutorial #8

Channel: Chai aur Code (Hitesh Choudhary)

Topic: forms.py, ModelChoiceField, CSRF, Views Wiring

1. Core Concept: Django Forms Ki Zaroorat Kyu Hai?

Ab tak humne dekha: **Models** (Data Layer) → **Views** (Logic Layer) → **Templates** (Presentation Layer). Lekin jab user se interactive data lena ho ya models ko forms ke roop me represent karna ho, tab introduce hota hai **Django Forms**.

- **Automatic HTML Rendering:** Django khud input fields, dropdowns, labels aur errors render karta hai.
- **Built-in Security & Validation:** Data type checking, cleaning, aur automatically malicious inputs ko filter karna.
- **Video Ka Practical Goal:** Ek drop-down form banana jisme Chai Varieties database se fetch hongy. Jab user koi Chai choose karega, toh server batayega ki woh specific chai kaun-kaun se stores par available hai.

2. Step-by-Step Practical Implementation

Step 1: forms.py Create Karna & ModelChoiceField Ka Use

App folder me ek nayi file banate hain: **chai/forms.py**. Yeh direct models ke data ko dropdown me bind kar deta hai:

```
from django import forms
from .models import ChaiVariety

class ChaiVarietyForm(forms.Form):
    # ModelChoiceField database ke objects se dynamic dropdown banata hai
    chai_variety = forms.ModelChoiceField(
        queryset=ChaiVariety.objects.all(),
        label="Select Chai Variety"
    )
```

Point to Remember: Agar manual text input lena ho toh `forms.CharField()` use hota hai, lekin database model ke data ko directly dropdown banana ho toh `ModelChoiceField(queryset=...)` best aur intelligent choice hai.

Step 2: views.py Me Form Handling (GET vs POST Lifecycle)

Django me Form handling ke hamesha **do main states** hote hain:

1. **GET Request:** Jab user pehli baar page kholta hai (Blank Form display karna).
2. **POST Request:** Jab user dropdown select karke form submit karta hai (Data extract & validate karna).

```
from django.shortcuts import render
from .models import ChaiVariety, Store
from .forms import ChaiVarietyForm

def chai_store_view(request):
    stores = None

    if request.method == 'POST':
        # 1. POST data ke saath form instantiate karo
        form = ChaiVarietyForm(request.POST)
        # 2. Validation check
        if form.is_valid():
            # 3. Cleaned data extract karo
            chai_variety = form.cleaned_data['chai_variety']
            # 4. Filter query: Wo stores dhoondo jahan ye chai available hai
            stores = Store.objects.filter(chai_varieties=chai_variety)
        else:
            # Pehli baar page khulne par blank form render karo
            form = ChaiVarietyForm()

    # Context me form aur stores dono ko template me pass karo
    return render(request, 'chai/chai_stores.html', {'form': form, 'stores': stores})
```

Step 3: urls.py Me Route Add Karna

File: **chai/urls.py**

```

from django.urls import path
from . import views

urlpatterns = [
    # chai/stores/ endpoint
    path('stores/', views.chai_store_view, name='chai_stores'),
]

```

Step 4: Template (chai_stores.html) Me Form & CSRF Token Render Karna

File: `chai/templates/chai/chai_stores.html`

```

{% extends 'layout.html' %}

{% block content %}
<h1>Find Stores by Chai Variety</h1>

<!-- Form must have method="POST" -->
<form method="POST">
    {% csrf_token %}          <!-- CRITICAL: CSRF Protection -->
    {{ form.as_p }}           <!-- Form fields ko paragraphs (<p>) me render karta hai -->
    <button type="submit">Search Store</button>
</form>

<!-- Conditional Rendering: Stores milne par display karna -->
{% if stores %}
    <h2>Stores Available:</h2>
    <ul>
        {% for store in stores %}
            <li><strong>{{ store.name }}</strong> - {{ store.location }}</li>
        {% endfor %}
    </ul>
{% endif %}
{% endblock %}

```

3. Important Core Concepts & Interview Highlights

⚠️ {% csrf_token %} Kyu Zaroori Hai? (CSRF Attack Protection)

Cross-Site Request Forgery (CSRF) ek common web vulnerability hai jisme unauthorized commands user ke browser se send kiye jaate hain. Django har POST form ke sath ek hidden csrfmiddlewaretoken bhejta hai. Agar template me {% csrf_token %} nahi likha toh Django **403 Forbidden: CSRF verification failed** error de dega.

Method / Property	Kaha Use Hota Hai?	Iska Asli Kaam Kya Hai? (Explanation in Hinglish)
request.POST	views.py	User ke dwara form submit karne par jo data payload request me aata hai.
form.is_valid()	views.py	Checks karta hai ki inputs Django form fields ke validation rules ko follow kar rahe hain ya nahi.
form.cleaned_data	views.py	Validation ke baad data ko clean dictionary format me return karta hai jise direct DB query me use kar sakte hain.
{{ form.as_p }}	HTML Template	Form ke labels aur input widgets ko standard <p> tags me wrap karke neat tarike se render karta hai.
ModelChoiceField	forms.py	Django model ka queryset lekar user ko HTML <select> (dropdown) bana ke deta hai.

4. Quick Revision Flowchart / Cheatsheet

- **Browser Request (GET):** User open karega /chai/stores/ \$ ightarrow\$ views.py ka else block chalega \$ ightarrow\$ Blank Form render hoga.
- **Form Submission (POST):** User chai select karke "Search Store" dabayega \$ ightarrow\$ request.method == 'POST' \$ ightarrow\$ ChaiVarietyForm(request.POST) me data aayega.
- **Validation & Filtering:** is_valid() pass hone ke baad cleaned_data['chai_variety'] se model match hoga \$ ightarrow\$ Store.objects.filter(...) chalega.

- **Template Response:** Template me {% if stores %} check hoga aur matching stores ke naam + locations list ho jayenge!